Practical - 1

Aim:
Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort Theory

Theory:
Bubble Sort repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order.

~~Selection S~~ort divides the array into sorted and unsorted regions. It repeatedly finds the smallest element from the unsorted part and moves it to the end of the sorted part.

~~Insertion Sort~~ constructs the final sorted array one element at a time. It picks an element from the unsorted portion and shifts sorted elements around to insert it into its correct position.

~~Merge Sort is~~ a divide-and-conquer algorithm. It recursively splits the array in half until individual sublists contain one element, then merges those sublists back together in sorted order.

Quicksort is another highly efficient divide-and-conquer algorithm. It picks an element as a "pivot" and partitions the array so that elements smaller than the pivot go to its left, and larger elements go to its right.

~~Algorithm~~

1. Bubble Sort
Start with the first element of the array.

Compare each pair of adjacent elements.

If the left element is greater than the right element, swap them.

Repeat the process for all adjacent pairs until the end of the array (one pass). After each pass, the largest element reaches its correct position.

Repeat the passes until no more swaps are needed or all elements are sorted.

2. Selection Sort

Assume the first unsorted element is the minimum.

Compare it with all remaining unsorted elements.

Find the smallest element in the unsorted portion.

Swap the smallest element with the first unsorted element.

Move the sorted boundary one position to the right and repeat until the array is sorted.

3. Insertion Sort

Consider the first element as already sorted.

Pick the next element (called the key).

Compare the key with elements in the sorted portion and shift larger elements one position to the right.
Insert the key into its correct position.

Repeat Steps 2–4 until all elements are inserted into the sorted portion.

4. Quick Sort

Choose a pivot element from the array.

Partition the array so that elements smaller than the pivot are placed on the left and larger elements on the right.
Place the pivot in its correct sorted position.

Recursively apply Quick Sort to the left subarray.

Recursively apply Quick Sort to the right subarray until each subarray contains one or zero

elements.

5. Merge Sort

Divide the array into two equal halves.

Recursively divide each half until each subarray contains only one element.

Compare the elements of two adjacent subarrays.

Merge the two subarrays into a single sorted subarray.

Continue merging all sorted subarrays until the complete array becomes sorted.

Program Code

```
#include <iostream>
#include <vector>
#include <cstdlib>
#include <ctime>
#include <chrono>

using namespace std;
using namespace std::chrono;

// Bubble Sort
void bubbleSort(vector<int> &arr)
{
   int n = arr.size();
   for(int i=0;i<n-1;i++)
      for(int j=0;j<n-i-1;j++)
         if(arr[j]>arr[j+1])
            swap(arr[j],arr[j+1]);
}

// Selection Sort
void selectionSort(vector<int> &arr)
{
   int n=arr.size();
```

```
    for(inti=0;i<n-1;i++)
    {
        intmin=i;

        for(intj=i+1;j<n;j++)
            if(arr[j]<arr[min])
                min=j;

        swap(arr[i],arr[min]);
    }
}

//InsertionSort
voidinsertionSort(vector<int> &arr)
{
    intn=arr.size();

    for(inti=1;i<n;i++)
    {
        intkey=arr[i];
        intj=i-1;

        while(j>=0 && arr[j]>key)
        {
            arr[j+1]=arr[j];
            j--;
        }

        arr[j+1]=key;
    }
}

//MergeSort
voidmerge(vector<int> &arr,int l,int m,int r)
{
    intn1=m-l+1;
    intn2=r-m;
```

```
vector<int> L(n1),R(n2);

   for(inti=0;i<n1;i++)
      L[i]=arr[l+i];

   for(intj=0;j<n2;j++)
      R[j]=arr[m+1+j];

   inti=0,j=0,k=l;

   while(i<n1 && j<n2)
   {
      if(L[i]<=R[j])
         arr[k++]=L[i++];
      else
         arr[k++]=R[j++];
   }

   while(i<n1)
      arr[k++]=L[i++];

   while(j<n2)
      arr[k++]=R[j++];
}

voidmergeSort(vector<int> &arr,int l,int r)
{
   if(l<r)
   {
      intm=(l+r)/2;

      mergeSort(arr,l,m);
      mergeSort(arr,m+1,r);
      merge(arr,l,m,r);

   }
```

```
}

// Quick Sort
intpartition(vector<int> &arr,int low,int high)
{
   intpivot=arr[high];
   inti=low-1;

   for(intj=low;j<high;j++)
   {
      if(arr[j]<pivot)
      {
         i++;
         swap(arr[i],arr[j]);
      }
   }

   swap(arr[i+1],arr[high]);

   return i+1;
}

voidquickSort(vector<int> &arr,int low,int high)
{
   if(low<high)
   {
      intpi=partition(arr,low,high);

      quickSort(arr,low,pi-1);
      quickSort(arr,pi+1,high);
   }
}

int main()
{
   constintn=100;
```

```
vector<int> arr(n),temp;

    srand(time(0));

    for(int i=0;i<n;i++)
        arr[i]=rand()%1000;

    cout<<"Number of Elements = "<<n<<"\n\n";

    auto start=high_resolution_clock::now();
    temp=arr;
    bubbleSort(temp);
    auto stop=high_resolution_clock::now();
    cout<<"Bubble Sort Time : "<<duration_cast<microseconds>(stop-start).count()<<"
microseconds\n";

    start=high_resolution_clock::now();
    temp=arr;
    selectionSort(temp);
    stop=high_resolution_clock::now();
    cout<<"Selection Sort Time : "<<duration_cast<microseconds>(stop-start).count()<<"
microseconds\n";

    start=high_resolution_clock::now();
    temp=arr;
    insertionSort(temp);
    stop=high_resolution_clock::now();
    cout<<"Insertion Sort Time : "<<duration_cast<microseconds>(stop-start).count()<<"
microseconds\n";

    start=high_resolution_clock::now();
    temp=arr;
    mergeSort(temp,0,n-1);
    stop=high_resolution_clock::now();
    cout<<"Merge Sort Time     : "<<duration_cast<microseconds>(stop-start).count()<<"
microseconds\n";
```

```
start=high_resolution_clock::now();
   temp=arr;
   quickSort(temp,0,n-1);
   stop=high_resolution_clock::now();
   cout<<"Quick Sort Time     : "<<duration_cast<microseconds>(stop-start).count()<<"
microseconds\n";

   cout<<"Srivek Annavarapu\n";
   cout<<"92460118171\n";
   cout<<"5-EN18\n";
   return 0;
}
```

Result :

```
Number of Elements = 100

Bubble Sort Time    : 111 microseconds
Selection Sort Time : 53 microseconds
Insertion Sort Time : 47 microseconds
Merge Sort Time     : 134 microseconds
Quick Sort Time     : 21 microseconds
Karthik Upadhya
92460118138
5-EN18

Process returned 0 (0x0)   execution time : 0.384 s
Press any key to continue.
```